RELAZIONE DI PROJECT WORK

Corso di Network and Cloud Infrastructures – A.A. 2025/2026
Docente: Prof. Giorgio Ventre

Informazioni degli Studenti

- **Candidato:** Pietroantonio De Rosa — **Matricola:** [DE9000126]
- **Repository GitHub:** `[https://github.com/giovannistanco/NCIS_unina/tree/main/2026):]`

---

1. Introduzione e Obiettivi del Progetto

Le Reti Software-Defined (SDN) rappresentano un cambiamento di paradigma fondamentale rispetto alle reti tradizionali, disaccoppiando il piano di controllo (*Control Plane*) dal piano dati (*Data Plane*). Questo approccio centralizzato consente una programmabilità avanzata della rete, facilitando la gestione, l'ottimizzazione del traffico e l'implementazione di politiche di sicurezza e monitoraggio dinamiche.

Scopo di questo progetto è la progettazione, l'implementazione e la validazione di un ambiente SDN funzionante basato su:

- **Mininet**: come framework di emulazione di rete per la creazione di una topologia personalizzata.
- **Ryu**: come controller SDN open-source per la gestione centralizzata dei flussi basata sul protocollo **OpenFlow 1.3**.
- **Modulo di Monitoraggio Personalizzato (`monitor_controller.py`)**: sviluppato in Python per l'estrazione periodica e l'analisi in tempo reale delle statistiche di porta (*Port Statistics*) e di flusso (*Flow Statistics*).

---

2. Architettura della Soluzione

L'architettura complessiva del sistema si articola su tre componenti principali:

1. **Ambiente di Esecuzione (WSL2 / Ubuntu):** Il progetto è stato sviluppato ed eseguito all'interno di un sottosistema Linux (Ubuntu su WSL2, Python 3.10), garantendo un

isolamento pulito tramite ambienti virtuali (venv) e la gestione delle dipendenze di rete con Open vSwitch (OVS).

2. **Topologia di Rete (topology.py):** È stata configurata una topologia a stella (*Star Topology*) gestita da un singolo switch OpenFlow 1.3 centrale (s1) a cui sono attestati tre host finali (h1, h2, h3), ciascuno configurato con indirizzamento IP nella sottorete 10.0.0.0/24.
3. **Logica di Controllo e Monitoraggio (monitor_controller.py):** Il controller Ryu interagisce con lo switch tramite OpenFlow 1.3. La componente di monitoraggio sfrutta thread asincroni (eventlet) per interrogare periodicamente lo switch (tramite messaggi OFPPortStatsRequest e OFPFlowStatsRequest), calcolando metriche fondamentali quali numero di pacchetti, byte transitati e throughput istantaneo (bps).

---

3. Implementazione del Codice Sviluppato

3.1 Topologia di Rete (`topology.py`)

La topologia è definita tramite le API Python di Mininet. Viene creata una struttura a stella con le seguenti caratteristiche:

- Switch centrale: `s1` (OpenFlow 1.3).
- Host connessi: `h1` (10.0.0.1), `h2` (10.0.0.2), `h3` (10.0.0.3).
- L'associazione con il controller remoto in ascolto sull'indirizzo di loopback `127.0.0.1` alla porta standard `6653`.

3.2 Controller di Monitoraggio (`monitor_controller.py`)

Il controller implementa una classe derivata da ryu.base.app_manager.RyuApp, registrando i listener per i messaggi asincroni di OpenFlow. Le caratteristiche chiave includono:

- **Polling Periodico:** Tramite un task in background (hub.spawn), il controller invia richieste di statistiche a intervalli regolari (es. ogni 5 secondi).
- **Gestione delle Risposte (StatsReply):** I gestori intercettano le risposte dello switch (_port_stats_reply_handler e _flow_stats_reply_handler), estraendo i contatori di traffico (rx_bytes, tx_bytes, packet_count, byte_count).
- **Calcolo del Throughput:** La velocità di trasmissione viene calcolata dinamicamente confrontando i campioni di byte accumulati nei successivi intervalli temporali Δt.

---

4. Ambiente di Simulazione e Risoluzione Criticità

L'ambiente è stato realizzato ed eseguito all'interno di **Ubuntu 22.04 su WSL2 (Windows Subsystem for Linux)**.

Durante la fase di setup sono state risolte diverse incompatibilità software legate all'ambiente Python 3.10:

- **Compatibilità Eventlet:** È stata forzata la versione `eventlet==0.33.3` per superare i problemi di immutabilità del tipo `TimeoutError` in Python 3.10.
- **Risoluzione Dipendenze:** Inserimento esplicito dei moduli `debtcollector`, `repoze.lru`, `oslo.config` per la stabilità del server WSGI di Ryu.

---

5. Risultati Sperimentali e Valutazione

Le prove di valutazione sono state effettuate generando traffico ICMP (`ping`) e UDP/TCP (`iperf`) tra gli host.

5.1 Test di Connettività

Eseguendo `h1 ping -c 10 h2`, si osserva una percentuale di **0% packet loss**, a conferma del corretto popolamento delle tabelle di flusso (*Flow Table*) da parte del controller Ryu.

Risultato: Connessione stabilita con successo, riscontro del **0% di packet loss** e tempi di latenza medi inferiori a $1ms$, confermando il corretto funzionamento dell'inoltro dei pacchetti e l'interazione con il controller.

5.2 Output del Monitoraggio

1. Stato Iniziale (A Riposo / Idle)

Mostra il comportamento a regime iniziale dello switch prima dell'invio di traffico utente.

Plaintext

```
None
================================================================================
==============================
 [FLOW MONITORING] - Switch DPID: 0000000000000001 | Rilevamento: 2026-11-06
16:33:45
================================================================================
==============================
```

```
Pri  In-Port   MAC Sorgente        MAC Destinazione    Azioni
Pacchetti  Byte         Throughput
-------------------------------------------------------------------------------
------------------------------
0    *         *                   *                   CONTROLLER         0
0 B          0.00 bps
-------------------------------------------------------------------------------
---------

================================================================================
================
 [PORT MONITORING] - Switch DPID: 0000000000000001 | Rilevamento: 2026-11-06
16:33:45
================================================================================
================
Porta  RX Pkts     TX Pkts     RX Bytes     TX Bytes     RX Throughput      TX
Throughput
-------------------------------------------------------------------------------
----------------
1      9           9           786 B        746 B        0.00 bps           0.00
bps
2      9           9           786 B        746 B        0.00 bps           0.00
bps
3      9           9           786 B        746 B        0.00 bps           0.00
bps
-------------------------------------------------------------------------------
---------
```

2. Durante il Test di Connettività (Ping)

Evidenzia l'installazione delle regole di flusso con priorità 1 tra i MAC address degli host e il calcolo dinamico del throughput in bassa intensità (bps).

Plaintext

```
None
================================================================================
==============================
 [FLOW MONITORING] - Switch DPID: 0000000000000001 | Rilevamento: 2026-11-06
16:34:00
================================================================================
==============================
Pri  In-Port   MAC Sorgente        MAC Destinazione    Azioni
Pacchetti  Byte         Throughput
-------------------------------------------------------------------------------
------------------------------
```

```
1    2         00:00:00:00:00:02   00:00:00:00:00:01   OUT:1             8
728 B        694.16 bps
1    1         00:00:00:00:00:01   00:00:00:00:00:02   OUT:2             7
630 B        694.16 bps
0    *         *                   *                   CONTROLLER        6
392 B        0.00 bps
--------------------------------------------------------------------------------
---------
```

3. Sotto Carico Elevato (Stress Test / iperf)

Mostra l'efficacia del sistema di monitoraggio sotto forte stress di rete, registrando un enorme volume di pacchetti e un throughput che scala dinamicamente fino a livelli di Gbps/Mbps.

Plaintext

```
None
================================================================================
==============================
 [FLOW MONITORING] - Switch DPID: 0000000000000001 | Rilevamento: 2026-11-06
16:35:10
================================================================================
==============================
Pri  In-Port   MAC Sorgente        MAC Destinazione    Azioni
Pacchetti  Byte         Throughput
--------------------------------------------------------------------------------
---------
1    1         00:00:00:00:00:01   00:00:00:00:00:02   OUT:2             200514
8.19 GB      14.07 Gbps
1    2         00:00:00:00:00:02   00:00:00:00:00:01   OUT:1             118697
7.47 MB      12.53 Mbps
0    *         *                   *                   CONTROLLER        9
602 B        0.00 bps
--------------------------------------------------------------------------------
---------
```

---

6. Conclusioni

Il progetto ha permesso di consolidare le competenze pratiche relative alle Reti Software-Defined, dimostrando l'efficacia del framework Ryu combinato con Mininet. Il

sistema di monitoraggio implementato fornisce una visibilità granulare in tempo reale dello stato della rete.

Possibili sviluppi futuri includono:

- L'estensione del monitoraggio a topologie multi-switch con calcolo automatico dei percorsi ottimali (routing basato su Dijkstra).
- L'integrazione di un modulo di machine learning o di soglie statistiche per il **rilevamento automatico di attacchi DoS/DDoS** basato sulle anomalie nel throughput.